УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 9

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ, СОДЕРЖАЩИХ ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ

ЛАБОРАТОРНАЯ РАБОТА № 9.1

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ, СОДЕРЖАЩИХ ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ С РАЗЛИЧНЫМИ ТИПАМИ АРГУМЕНТОВ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ, содержащих пользовательские функции с различными типами аргументов

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

Функция – это фрагмент кода, который можно вызвать из любого места программы. Например, с помощью функции **len()** можно получить количество элементов последовательности. Начиная с этого шага, рассмотрим создание пользовательских функций, которые позволят уменьшить избыточность программного кода и повысить его структурированность.

4.1 Определение и вызов функции

Функция создается (или, как говорят программисты, определяется) с помощью ключевого слова **def** по следующей схеме:

```
def <Имя функции> ( [ <Параметры> ] ) :
    ["""  Строка документирования  """]
    <Тело функции>
    [return <Результат>]
```

Имя функции должно быть уникальным идентификатором, состоящим из латинских букв, цифр и знаков подчеркивания, причем имя функции не может начинаться с цифры. В качестве имени нельзя использовать ключевые слова, кроме того, следует избегать совпадений с названиями встроенных идентификаторов. Регистр символов в названии функции также имеет значение.

После имени функции в круглых скобках можно указать один или несколько параметров через запятую, а если функция не принимает параметры, указываются только круглые скобки. После круглых скобок ставится двоеточие.

Тело функции представляет собой составную конструкцию. Как и в любой составной конструкции, инструкции внутри функции выделяются одинаковым количеством пробелов слева. Концом функции считается инструкция, перед

которой находится меньшее количество пробелов. Если тело функции не содержит инструкций, то внутри ее необходимо разместить оператор **pass**, который не выполняет никаких действий. Этот оператор удобно использовать на этапе отладки программы, когда определили функцию, а тело решили дописать позже.

Пример функции, которая ничего не делает:

```
def  func(): pass
```

Необязательная инструкция **return** позволяет вернуть из функции какое-либо значение в качестве результата. После исполнения этой инструкции выполнение функции будет остановлено. Это означает, что инструкции, следующие после оператора **return**, никогда не будут выполнены.

Пример:

```
def func():
    print("Текст до инструкции return")
    return "Возвращаемое значение"
    print("Эта инструкция никогда не будет выполнена")
print(func()) # Вызываем функцию
```

Результат выполнения:

```
Текст до инструкции return
Возвращаемое значение
```

Инструкции **return** может не быть вообще. В этом случае выполняются все инструкции внутри функции, и в качестве результата возвращается значение **None**.

Для примера создадим три функции:

```
def print_ok():
    """ Пример функции без параметров """
    print("Сообщение при удачно выполненной операции")
def echo(m):
    """ Пример функции с параметром """
    print(m)
def summa(x, y):
    """ Пример функции с параметрами,
        возвращающей сумму двух переменных """
    return x + y
```

При вызове функции значения передаются внутри круглых скобок через запятую. Если функция не принимает параметров, то указываются только круглые скобки. Необходимо также заметить, что количество параметров в определении функции должно совпадать с количеством параметров при вызове, иначе будет выведено сообщение об ошибке. Вызвать созданные функции можно способами, указанными ниже:

```
print_ok()          # Вызываем функцию без параметров
echo("Сообщение")      #  Функция  выведет  сообщение
x = summa(5, 2) # Переменной x будет присвоено значение 7
a, b = 10, 50
y = summa(a, b) # Переменной y будет присвоено значение 60
```

Как видно из последнего примера, имя переменной в вызове функции может не совпадать с именем переменной в определении функции. Кроме того,

глобальные переменные **x** и **y** не конфликтуют с одноименными переменными в определении функции, т. к. они расположены в разных областях видимости. Переменные, указанные в определении функции, являются локальными и доступны только внутри функции.

Оператор +, используемый в функции **summa()**, применяется не только для сложения чисел, но и позволяет объединить последовательности. То есть, функция **summa ()** может использоваться не только для сложения чисел. В качестве примера выполним конкатенацию строк и объединение списков:

```
def summa(x, y):
    return x + y
print(summa("str", "ing"))   # Выведет: string
print (summa ( [1, 2], [3, 4])) # Выведет: [1, 2, 3, 4]
```

Все в языке Python представляет собой объекты: строки, списки и даже сами типы данных. Не являются исключением и функции. Инструкция **def** создает объект, имеющий тип **function**, и сохраняет ссылку на него в идентификаторе, указанном после инструкции **def**. Таким образом, можно сохранить ссылку на функцию в другой переменной - для этого название функции указывается без круглых скобок. Сохраним ссылку в переменной и вызовем функцию через нее:

```
def summa (x, y) :
    return  x + y
f = summa # Сохраняем ссылку в переменной f
v = f(10, 20) #   Вызываем функцию через переменную f
```

Можно также передать ссылку на функцию в качестве параметра другой функции. Функции, передаваемые по ссылке, обычно называются **функциями обратного вызова**:

```
def  summa(x, y):
    return x  +  y
def  func(f, a, b):
    """ Через переменную f будет доступна ссылка на
        функцию summa() """
    return f (a, b) # Вызываем функцию summa ()
# Передаем ссылку на функцию в качестве параметра
v = func(summa, 10, 20)
```

Объекты функций поддерживают множество атрибутов, обратиться к которым можно, указав атрибут после названия функции через точку. Например, через атрибут **__name__** можно получить название функции в виде строки, через атрибут __doc__ - строку документирования и т. д. Для примера выведем названия всех атрибутов функции с помощью встроенной функции **dir()**:

```
def summa(x, y) :
 """ Суммирование двух чисел """
            return x + y
dir (summa)
['__annotations__', '__call__', '__class__', '__closure__', '__code__',
'__defaults__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__get__', '__getattribute__', '__globals__',
```

```
'__gt__', '__hash__', '__init__', '__kwdefaults__', '__le__', '__lt__',
'__module__', '__name__', '__ne__', '__new__', '__qualname__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__']
print(summa.__code__.co_varnames)
('x', 'y')
print(summa.__doc_)
' Суммирование двух чисел '
```

4.2 Расположение определений функций

Все инструкции в программе выполняются последовательно сверху вниз. Это означает, что, прежде чем использовать в программе идентификатор, его необходимо предварительно определить, присвоив ему значение. Поэтому определение функции должно быть расположено перед вызовом функции.

Правильно:

```
def summa(x, y):
    return x + y
v = summa(10, 20) # Вызываем после определения. Все нормально
```

Неправильно:

```
v = summa(10, 20)  # Идентификатор еще не определен. Это ошибка!!!
def summa(x, y):
    return x + y
```

В последнем случае будет выведено сообщение об ошибке: **NameError: name 'summa' is not defined**. Чтобы избежать ошибки, определение функции размещают в самом начале программы после подключения модулей или в отдельном модуле.

С помощью оператора ветвления **if** можно изменить порядок выполнения программы - например, разместить внутри условия несколько определений функций с одинаковым названием, но разной реализацией:

```
# -*- coding: utf-8  -*-
n = input("Введите 1 для вызова первой функции: ")
if n == "1":
    def echo():
        print("Вы ввели число 1")
else:
    def echo():
        print("Альтернативная функция")
echo() # Вызываем функцию
input()
```

При вводе числа 1 мы получим сообщение ***"Вы ввели число 1"***, в противном случае - ***"Альтернативная функция"***.

Помните, что инструкция **def** всего лишь присваивает ссылку на объект функции идентификатору, расположенному после ключевого слова **def**. Если

определение одной функции встречается в программе несколько раз, то будет использоваться функция, которая была определена последней. Пример:

```
def echo():
    print("Вы ввели число 1")
def echo():
    print("Альтернативная функция")
echo() # Всегда выводит "Альтернативная функция"
```

4.3 Необязательные параметры и сопоставление по ключам

Чтобы сделать некоторые параметры необязательными, следует в определении функции присвоить этому параметру начальное значение. Переделаем функцию суммирования двух чисел и сделаем второй параметр необязательным.

```
def summa(x, y=2): # y - необязательный параметр
    return x + y
a = summa(5) # Переменной a будет присвоено значение 7
b = summa(10, 50) # Переменной b будет присвоено значение 60
```

Таким образом, если второй параметр не задан, то его значение будет равно 2. Обратите внимание на то, что необязательные параметры должны следовать после обязательных параметров, иначе будет выведено сообщение об ошибке.

До сих пор мы использовали позиционную передачу параметров в функцию:

```
def summa(x, y):
    return x + y
print(summa(10, 20)) # Выведет: 30
```

Переменной **x** при сопоставлении будет присвоено значение 10, а переменной **y** - значение 20. Но язык **Python** позволяет также передать значения в функцию, используя сопоставление по ключам. Для этого при вызове функции параметрам присваиваются значения, причем последовательность указания параметров в этом случае может быть произвольной:

```
def summa(x, y):
    return x + y
print(summa(y=20, x=10))  # Сопоставление по ключам
```

Сопоставление по ключам очень удобно использовать, если функция имеет несколько необязательных параметров. В этом случае не нужно перечислять все значения, а достаточно присвоить значение нужному параметру. Пример:

```
def summa(a=2, b=3, c=4): # Все параметры являются необязательными
    return a + b + c
print(summa(2, 3, 20)) # Позиционное присваивание
print(summa(c=20)) # Сопоставление по ключам
```

Если значения параметров, которые планируется передать в функцию, содержатся в кортеже или списке, то перед объектом следует указать символ *. Пример передачи значений из кортежа и списка приведен ниже:

```
def summa(a, b, c):
```

```
    return a + b + c
t1, arr = (1, 2, 3), [1, 2, 3]
print (summa(*t1)) # Распаковываем кортеж
print(summa(*arr)) # Распаковываем список
t2 = (2, 3)
print(summa(1, *t2)) # Можно комбинировать значения
```

Если значения параметров содержатся в словаре, то распаковать словарь можно, указав перед ним две звездочки (**):

```
def summa(a, b, c):
    return a + b + c
d1 = {"a": 1, "b": 2, "c": 3}
print(summa(**d1)) # Распаковываем словарь
t, d2 = (1, 2), {"c": 3}
print(summa(*t, **d2)) # Можно комбинировать значения
```

Объекты в функцию передаются по ссылке. Если объект относится к неизменяемому типу, то изменение значения внутри функции не затронет значение переменной вне функции:

```
def func(a, b):
    a, b = 20, "str"
x, s = 80, "test"
func(x, s) # Значения переменных x и s не изменяются
print(x, s) # Выведет: 80 test
```

В этом примере значения в переменных **x** и **s** не изменились. Однако если объект относится к изменяемому типу, то ситуация будет другой:

```
def func (a, b):
    a[0], b["a"] = "str", 800


x = [1, 2, 3] # Список
y = {"a": 1, "b": 2} #  Словарь
func(x, y) # Значения будут изменены!!!
print (x, y) # Выведет: ['str', 2, 3] {'a': 800, 'b': 2}
```

Как видно из примера, значения в переменных **x** и **y** изменились, поскольку список и словарь относятся к изменяемым типам. Чтобы избежать изменения значений, внутри функции следует создать копию объекта:

```
def func (a, b):
    a = a[:] # Создаем поверхностную копию списка
    b = b.copy() # Создаем поверхностную копию словаря
    a[0], b["a"] = "str", 800
x = [1, 2, 3] # Список
y = {"a": 1, "b": 2} # Словарь
func(x, y) # Значения останутся прежними
print (x, y) # Выведет: [1, 2, 3] {'a': 1, 'b': 2}
```

Можно также сразу передавать копию объекта в вызове функции:

```
 func(x[:], y.copy())
```

Если указать объект, имеющий изменяемый тип, в качестве значения по умолчанию, то этот объект будет сохраняться между вызовами функции. Пример:

```
def func(a=[]):
    a.append(2)
    return a
print(func()) # Выведет: [2]
print(func()) # Выведет: [2, 2]
print(func()) # Выведет: [2, 2, 2]
```

Как видно из примера, значения накапливаются внутри списка. Обойти эту проблему можно, например, следующим образом:

```
def func(a=None): # Создаем новый список, если значение равно None
    if a is None:
        a = []
    a.append(2)
    return a
print(func())    # Выведет: [2]
print(func([1])) # Выведет: [1, 2]
print(func())    # Выведет: [2]
```

4.4 Переменное число параметров в функции

Если перед параметром в определении функции указать символ *****, то функции можно будет передать произвольное количество параметров. Все переданные параметры сохраняются в кортеже. Для примера напишем функцию суммирования произвольного количества чисел:

```
def summa(*t):
    """ Функция принимает произвольное количество параметров """
    res = 0
    for i in t: # Перебираем кортеж с переданными параметрами
        res += i
    return res
print(summa(10, 20)) # Выведет: 30
print(summa(10, 20, 30, 40, 50, 60)) # Выведет: 210
```

Можно также вначале указать несколько обязательных параметров и параметров, имеющих значения по умолчанию:

```
def summa(x, y=5, *t): # Комбинация параметров
    res = x + y
    for i in t: # Перебираем кортеж с переданными параметрами
        res += i
    return res
print(summa(10)) # Выведет: 15
print(summa(10, 20, 30, 40, 50, 60)) # Выведет: 210
```

Если перед параметром в определении функции указать две звездочки (******), то все именованные параметры будут сохранены в словаре:

```
def func(**d):
```

```
    for i in d:       # Перебираем словарь с переданными параметрами
        print("{0} => {1}".format(i, d[i]), end=" ")
func(a=1, b=2, c=3)  # Выведет: a => 1 c => 3 b => 2
```

При комбинировании параметров параметр с двумя звездочками указывается самым последним, Если в определении функции указывается комбинация параметров с одной звездочкой и двумя звездочками, то функция примет любые переданные ей параметры:

```
def func(*t, **d):
    """ Функция примет любые параметры """
    for i in t:
        print(i, end=" ")
    for i in d: # Перебираем словарь с переданными параметрами
        print ("{0} => {1}".format(i, d[i]), end=" ")
func(35, 10, a=1, b=2, c=3) # Выведет: 35 10 a => 1 c => 3 b => 2
func(10) # Выведет: 10
func(a=1, b=2) # Выведет: a => 1 b => 2
```

В определении функции можно указать, что некоторые параметры передаются только по именам. Для этого параметры должны указываться после параметра с одной звездочкой, но перед параметром с двумя звездочками. Именованные параметры могут иметь значения по умолчанию. Пример:

```
def func(*t, a, b=10, **d):
    print(t, a, b, d)
func(35, 10, a=1, c=3) # Выведет: (35, 10) 1 10 {'c': 3}
func(10, a=5) # Выведет: (10,) 5 10 {}
func(a=1, b=2) # Выведет: () 1 2 {}
func(1, 2, 3) # Ошибка. Параметр a обязателен!
```

В этом примере переменная **t** примет любое количество значений, которые будут объединены в кортеж. Переменные **a** и **b** должны передаваться только по именам, причем переменной **a** обязательно нужно передать значение при вызове функции. Переменная **b** имеет значение по умолчанию, поэтому при вызове допускается не передавать ей значение, но если значение передается, то оно должно быть указано после названия параметра и оператора =. Переменная **d** примет любое количество именованных параметров и сохранит их в словаре. Обратите внимание на то, что, хотя переменные **a** и **b** являются именованными, они не попадут в этот словарь.

Параметра с двумя звездочками может не быть в определении функции, а вот параметр с одной звездочкой при указании параметров, передаваемых только по именам, должен присутствовать обязательно. Если функция не должна принимать переменного количества параметров, но необходимо использовать переменные, передаваемые только по именам, то можно указать только звездочку без переменной:

```
def func(x=1, y=2, *, a, b=10):
    print(x, y, a, b)
func(35, 10, a=1) # Выведет: 35 10 1 10
func(10, a=5) # Выведет: 10 2 5 10
```

```
func(a=1, b=2) # Выведет: 1 2 1 2
func(a=1, y=8, x=7) # Выведет: 7 8 1 10
func(1, 2, 3) # Ошибка. Параметр a обязателен!
```

В этом примере значения переменным **x** и **y** можно передавать как по позициям, так и по именам. Поскольку переменные имеют значения по умолчанию, допускается вообще не передавать им значений. Переменные **a** и **b** расположены после параметра с одной звездочкой, поэтому передать значения при вызове можно только по именам. Так как переменная **b** имеет значение по умолчанию, допускается не передавать ей значение при вызове, а вот переменная **a** обязательно должна получить значение, причем только по имени.

4.5 Глобальные и локальные переменные

Глобальные переменные – это переменные, объявленные в программе вне функции. В Python глобальные переменные видны в любой части модуля, включая функции.

```
def func(glob2):
    print("Значение глобальной переменной glob1 =", glob1)
    glob2 +=10
    print("Значение локальной переменной glob2 =", glob2)
glob1, glob2 = 10, 5
func(77)  #  Вызываем функцию
print("Значение глобальной переменной glob2 =", glob2)
```

Переменной **glob2** внутри функции присваивается значение, переданное при вызове функции. В результате создается новая переменная с тем же именем: **glob2**, которая является локальной. Все изменения этой переменной внутри функции не затронут значение одноименной глобальной переменной.

Итак, **локальные переменные** — это переменные, объявляемые внутри функций. Если имя локальной переменной совпадает с именем глобальной переменной, то все операции внутри функции осуществляются с локальной переменной, а значение глобальной переменной не изменяется. Локальные переменные видны только внутри тела функции.

```
def func():
    local1 = 77     # Локальная переменная
    glob1 = 25      # Локальная переменная
    print("Значение glob1 внутри функции =", glob1)
glob1 = 10 # Глобальная переменная
func() # Вызываем функцию
print("Значение glob1 вне функции =", glob1)
try:
    print(local1) # Вызовет исключение NameError
except NameError: # Обрабатываем исключение
    print("Переменная local1 не видна вне функции")
```

Как видно из примера, переменная **local1**, объявленная внутри функции **func()**, недоступна вне функции. Объявление внутри функции локальной переменной **glob1** не изменило значения одноименной глобальной переменной.

Если обращение к переменной производится до присваивания значения (даже если существует одноименная глобальная переменная), то будет возбуждено исключение **UnboundLocalError**.

```
def func():
    # Локальная переменная еще не определена
    print(glob1) # Эта строка вызовет ошибку!!!
    glob1 = 25      # Локальная переменная
glob1 = 10 # Глобальная переменная
func() # Вызываем функцию
```

Для того чтобы значение глобальной переменной можно было изменить внутри функции, необходимо объявить переменную глобальной с помощью ключевого слова **global**. Продемонстрируем это на примере.

```
def func():
    # Объявляем переменную glob1 глобальной
    global glob1
    glob1 = 25 # Изменяем значение глобальной переменной
    print("Значение glob1 внутри функции =", glob1)
glob1 = 10 # Глобальная переменная
print("Значение glob1 вне функции =", glob1)
func() # Вызываем функцию
print("Значение glob1 после функции =", glob1)
```

Таким образом, поиск идентификатора, используемого внутри функции, будет производиться в следующем порядке:

– поиск объявления идентификатора внутри функции (в локальной области видимости);

– поиск объявления идентификатора в глобальной области;

– поиск во встроенной области видимости (встроенные функции, классы и т. д.).

При использовании анонимных функций следует учитывать, что при указании внутри функции глобальной переменной будет сохранена ссылка на эту переменную, а не ее значение в момент определения функции:

```
x = 5
# Сохраняется ссылка, а не значение переменной x!
func = lambda: x
x =  80   # Изменили значение
print (func ()) #  Выведет: 80, а не 5
```

Если необходимо сохранить именно текущее значение переменной, то можно воспользоваться способом, приведенным ниже.

```
x = 5
# Сохраняется значение переменной x
func = (lambda y: lambda: y)(x)
x = 80    # Изменили значение
```

```
print(func()) # Выведет: 5
```

В третьей строке примера была определена анонимная функция с одним параметром, возвращающая ссылку на вложенную анонимную функцию. Далее вызывается первая функция с помощью круглых скобок и передаем ей значение переменной **x**. В результате сохраняется текущее значение переменной, а не ссылка на нее.

Сохранить текущее значение переменной можно также, указав глобальную переменную в качестве значения параметра по умолчанию в определении функции.

```
x = 5
# Сохраняется значение  переменной x
func = lambda  x=x: x
x = 80    # Изменили значение
print(func()) # Выведет: 5
```

Получить все идентификаторы и их значения позволяют следующие функции:

- **globals ()** - возвращает словарь с глобальными идентификаторами;
- **locals ()** - возвращает словарь с локальными идентификаторами.

Пример:

```
def func():
    local1 = 54
    glob2 = 25
    print("Глобальные идентификаторы внутри функции")
    print (sorted(globals().keys()))
    print ("Локальные идентификаторы внутри функции")
    print(sorted(locals().keys()))
glob1, glob2 = 10, 88
func()
print("Глобальные идентификаторы вне функции")
print(sorted(globals().keys()))
```

vars ([<Объект>]) - если вызывается без параметра внутри функции, то возвращает словарь с локальными идентификаторами. Если вызывается без параметра вне функции, то возвращает словарь с глобальными идентификаторами. При указании объекта в качестве параметра возвращает идентификаторы этого объекта (эквивалентно вызову <Объект>.__dict__). Пример:

```
def func():
    local1 = 54
    glob2 = 25
    print ("Локальные идентификаторы внутри функции")
    print (sorted(vars().keys()))
glob1, glob2 = 10, 88
func()
print("Глобальные идентификаторы вне функции")
print(sorted(vars().keys()))
print("Указание объекта в качестве параметра")
```

```
print(sorted(vars(dict).keys()))
print("Альтернативный вызов")
print(sorted(dict.__dict__.keys()))
```

4.6 Пример решения задачи на языке программирования Python

Задача. Вводится последовательность вещественных чисел. Известно, что последний элемент последовательности равен 5. Разработайте функцию, подсчитывающую количество положительных чисел и минимальное из них.

Решение задачи:

```
def func(chislo=0):
    kolpol=0
    min_=32767
    while chislo!=5: #Пока число, введенное в цикле, не равно числу 5, выполнять тело цикла
        chislo = float(input("Введите число "))
        if chislo>0: #Проверка на положительность очередного введенного числа
            kolpol=kolpol+1
            if chislo<min_: #Реализация алгоритма поиска минимального элемента
                min_ = chislo
    return kolpol,min_
k,m=func()
print("Количество положительных чисел =" , k)
print("Минимальное из них =" , m)
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите функцию, которая вычисляет среднее арифметическое элементов списка, переданного ей в качестве аргумента.

2 Напишите функцию, которая определяет количество цифр в заданном целом числе.

3 Напишите функцию, которая принимает список чисел и возвращает среднее геометрическое его элементов.

4 Напишите функцию, реализующую бинарный поиск элемента в упорядоченном по возрастанию списке и возвращающую его позицию либо сообщение об отсутствии элемента.

5 Напишите функцию, которая находит наибольший общий делитель (НОД) двух заданных чисел.

6 Напишите функцию для вычисления площади круга, функцию для вычисления площади прямоугольника и функцию для вычисления площади треугольника. Выбор необходимой функции осуществляется пользователем.

7 Напишите функцию, которая вычисляет сумму элементов списка. Используйте её для определения массива с максимальной суммой среди десяти сгенерированных массивов случайных чисел.

8 Напишите функцию, которая вычисляет сумму элементов главной или побочной диагонали квадратной матрицы в зависимости от переданного параметра.

9 Напишите функцию, которая переводит десятичное число в двоичную систему счисления.

10 Напишите функцию, которая принимает число и возвращает значение True, если оно является степенью двойки, и False в противном случае.

11 Напишите функцию, которая подсчитывает количество гласных и согласных букв в заданной строке.

12 Напишите функцию, которая принимает два списка и возвращает список их общих элементов.

13 Напишите функцию, которая проверяет, является ли заданная строка палиндромом.

14 Напишите функцию, которая принимает список чисел и возвращает наибольший и наименьший элементы этого списка.

15 Напишите функцию, которая принимает строку и возвращает количество слов, содержащихся в ней.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите функцию, которая принимает два числа и возвращает результат их сложения.

2 Напишите функцию, которая принимает список чисел и вычисляет произведение всех его элементов.

3 Напишите функцию, которая принимает строку и возвращает её в обратном порядке.

4 Напишите функцию, которая принимает список строк и формирует новый список, содержащий только строки длиной более 5 символов.

5 Напишите функцию, которая определяет, является ли заданное число простым.

6 Напишите функцию, которая принимает список чисел и возвращает новый список, состоящий только из чётных элементов.

7 Напишите функцию, которая подсчитывает количество гласных букв в заданной строке.

8 Напишите функцию, которая вычисляет среднее арифметическое элементов списка.

9 Напишите функцию, которая находит факториал заданного числа.

10 Напишите функцию, которая формирует список уникальных элементов на основе исходного списка.

11 Напишите функцию, которая принимает список чисел и возвращает список простых чисел, содержащихся в нём.

12 Напишите функцию, которая проверяет, является ли заданная строка палиндромом.

13 Напишите функцию, которая принимает два списка и возвращает список элементов, присутствующих в обоих списках.

14 Напишите функцию, которая принимает список чисел и возвращает сумму всех отрицательных элементов этого списка.

15 Напишите функцию, которая принимает строку и возвращает количество слов, содержащихся в этой строке.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Дайте определение понятию «Функция» в языке программирования Python.

7.2 Приведите синтаксис объявления функций в языке программирования Python.

7.3 Объясните расположение определений функций в языке программирования Python.

7.4 Опишите использование глобальных и локальных переменных в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.